

COMPONENTES SWING – REFERENCIA RÁPIDA

Componente	Clase	Uso
Ventana	<code>JFrame</code>	Contenedor principal de la app
Botón	<code>JButton</code>	Acción al hacer clic
Campo de texto	<code>JTextField</code>	Entrada de una línea
Etiqueta	<code>JLabel</code>	Texto estático no editable
Área de texto	<code>JTextArea</code>	Entrada de múltiples líneas
Panel	<code>JPanel</code>	Contenedor para agrupar componentes
Diálogo mensaje	<code>JOptionPane</code>	Ventanas emergentes de aviso
Casilla	<code>JCheckBox</code>	Opción booleana marcabale
Lista desplegable	<code>JComboBox</code>	Seleccionar una opción de varios

MÉTODOS CLAVE DE JFRAME

<code>setSize(w,h)</code> ancho y alto en píxeles	<code>setVisible(true)</code> muestra la ventana
<code>setTitle("txt")</code> título de la barra	<code>setDefaultCloseOperation(EXIT_ON_CLOSE)</code> cierra la app al cerrar ventana
<code>add(comp)</code> añade componente a la ventana	<code>setLayout(layout)</code> establece el gestor de layout
<code>pack()</code> ajusta el tamaño al contenido	<code>setLocationRelativeTo(null)</code> centra la ventana en pantalla

LAYOUTS DISPONIBLES

Layout	Comportamiento
<code>FlowLayout</code>	Fila izq→der, salta a nueva línea si no hay espacio. Default de JPanel.
<code>BorderLayout</code>	5 zonas: NORTH, SOUTH, EAST, WEST, CENTER. Default de JFrame.
<code>GridLayout(f,c)</code>	Cuadrícula de f filas × c columnas, todas iguales.
<code>BoxLayout</code>	Apila componentes vertical u horizontalmente.

ERRORES COMUNES

- No llamar a `setVisible(true)` → la ventana se crea pero no aparece
- `Integer.parseInt(campo.getText())` sin try/catch → `NumberFormatException` si el campo está vacío o tiene letras
- Añadir componentes después de `setVisible(true)` sin llamar a `revalidate()` → no aparecen
- Sin layout o sin `JPanel` → solo el último componente añadido con `add()` es visible en `JFrame` con `BorderLayout`

EJEMPLO COMPLETO – CALCULADORA DE SUMA

Patrón más completo del bloque: dos campos de texto, un botón con lambda, lógica de negocio y visualización del resultado con `JOptionPane`.

```
import javax.swing.*;

public class Calculadora {
    public static void main(String[] args) {
        JFrame ventana = new JFrame("Calculadora");
        JTextField num1 = new JTextField(5);
        JTextField num2 = new JTextField(5);
        JButton boton = new JButton("Sumar");

        boton.addActionListener(e → {
            try {
                int a = Integer.parseInt(num1.getText());
                int b = Integer.parseInt(num2.getText());
                JOptionPane.showMessageDialog(
                    null, "Resultado: " + (a + b));
            } catch (NumberFormatException ex) {
                JOptionPane.showMessageDialog(
                    null, "Introduce números válidos");
            }
        });

        JPanel panel = new JPanel(); // FlowLayout
        panel.add(num1); panel.add(num2); panel.add(boton);
        ventana.add(panel);
        ventana.setSize(300, 200);
        ventana.setDefaultCloseOperation(
            JFrame.EXIT_ON_CLOSE);
        ventana.setVisible(true);
    }
}
```

JOPTIONPANE – DIÁLOGOS RÁPIDOS

MOSTRAR MENSAJE	
<code>JOptionPane.showMessageDialog(null, "Hola!");</code>	
PEDIR UN DATO	
<code>String nombre = JOptionPane.showInputDialog("Tu nombre:");</code>	
CONFIRMAR ACCIÓN	
<code>int res = JOptionPane.showConfirmDialog(null, "¿Estás seguro?"); if (res == JOptionPane.YES_OPTION) { /* si */ }</code>	
Método	Devuelve
<code>showMessageDialog()</code>	void — solo muestra
<code>showInputDialog()</code>	String — lo que escribe el usuario
<code>showConfirmDialog()</code>	int — YES/NO/CANCEL_OPTION

Eventos y GUI

BLOQUE 5 · SWING · CHULETARIO · TRÍPTICO A4

MODELO DE EVENTOS



TIPOS DE EVENTOS

Evento	Listener	Fuente típica
<code>ActionEvent</code>	<code>ActionListener</code>	JButton, JTextField (Enter)
<code>MouseEvent</code>	<code>MouseListener</code>	Clic, arrastre sobre componente
<code>KeyEvent</code>	<code>KeyListener</code>	Pulsaciones de teclado
<code>WindowEvent</code>	<code>WindowListener</code>	Abrir / cerrar ventana

CONTENIDOS DE ESTE TRÍPTICO

- Interior izq. — Eventos, JFrame, JButton, ActionListener clásico
- Interior cent. — Lambda, JTextField, JPanel, layouts
- Interior der. — Tabla de componentes, buenas prácticas, resumen
- Solapa — Calculadora completa, JOptionPane (los 3 tipos)
- Contraportada — Todos los componentes, métodos JFrame, errores comunes

CARA A → Exterior · CARA B → Interior (teoría y referencia)
Imprimir: A4 horizontal · doble cara · voltear por lado largo

PROGRAMACIÓN DIRIGIDA POR EVENTOS

El programa **no ejecuta instrucciones en orden fijo**. Espera a que ocurra un **evento** (clic, tecla, cierre...) y reacciona ejecutando el código del *listener* asociado.

Concepto	Descripción
Evento	Lo que ocurre: clic, tecla, cierre...
Source	El componente que genera el evento
Listener	Objeto que escucha y reacciona al evento

JFRAME – VENTANA BÁSICA

```
import javax.swing.JFrame;

public class VentanaBasica {
    public static void main(String[] args) {
        JFrame ventana = new JFrame("Mi ventana");
        ventana.setSize(400, 300);
        ventana.setDefaultCloseOperation(
            JFrame.EXIT_ON_CLOSE);
        ventana.setVisible(true);
    }
}
```

Código	Qué hace
new JFrame("titulo")	Crea la ventana con título en la barra
setSize(400,300)	Anchura × altura en píxeles
EXIT_ON_CLOSE	Cierra la JVM al cerrar la ventana
setVisible(true)	Hace la ventana visible (siempre al final)

ACTIONLISTENER – FORMA CLÁSICA

Se implementa con una clase anónima que sobrescribe `actionPerformed()`. Es la forma tradicional (Java < 8).

```
import javax.swing.*;
import java.awt.event.*;

JButton boton = new JButton("Pulsar");

boton.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        System.out.println("Botón pulsado");
    }
});
```

Elemento	Rol
addActionListener()	Registra el listener en el componente
ActionListener	Interfaz con método <code>actionPerformed()</code>
ActionEvent e	Objeto con info del evento (fuente, hora...)
e.getSource()	Devuelve el componente que generó el evento

ACTIONLISTENER CON LAMBDA – FORMA MODERNA

Java 8+. `ActionListener` es una interfaz funcional (un solo método). Se puede reemplazar por una expresión lambda, mucho más concisa.

```
// Forma clásica (clase anónima)
boton.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        System.out.println("Clic");
    }
});

// Forma moderna (lambda) + usar siempre
boton.addActionListener(e →
    System.out.println("Clic"));

// Lambda con varias instrucciones
boton.addActionListener(e → {
    String txt = campo.getText();
    System.out.println(txt);
});
```

JTEXTFIELD – CAMPO DE TEXTO

```
JTextField campo = new JTextField(15); // 15 cols

// Leer lo que escribió el usuario
String texto = campo.getText();

// Escribir texto desde código
campo.setText("Valor inicial");

// Vaciar el campo
campo.setText("");
```

JPANEL + AÑADIR COMPONENTES

`JPanel` es un contenedor ligero. Agrupa componentes y aplica `FlowLayout` por defecto. Siempre añade los componentes al panel, y el panel a la ventana.

```
JPanel panel = new JPanel(); // FlowLayout por defecto
panel.add(campo);
panel.add(boton);
ventana.add(panel); // panel → ventana
```

LAYOUTS – ORGANIZAR COMPONENTES

FLOWLAYOUT (DEFECTO JPANEL)
JPanel p = new JPanel(); // ya usa FlowLayout
BORDERLAYOUT (DEFECTO JFRAME)
ventana.add(norte, BorderLayout.NORTH); ventana.add(centro, BorderLayout.CENTER); ventana.add(sur, BorderLayout.SOUTH);
GRIDLAYOUT – CUADRÍCULA UNIFORME
JPanel p = new JPanel(new GridLayout(2, 3)); // 2 filas × 3 columnas, todas las celdas iguales

APP CON CAMPO DE TEXTO Y BOTÓN

```
import javax.swing.*;

public class CampoTexto {
    public static void main(String[] args) {
        JFrame ventana = new JFrame("Texto");
        JTextField campo = new JTextField(15);
        JButton boton = new JButton("Mostrar");

        boton.addActionListener(e →
            System.out.println(campo.getText()));

        JPanel panel = new JPanel();
        panel.add(campo); panel.add(boton);
        ventana.add(panel);
        ventana.setSize(300, 150);
        ventana.setDefaultCloseOperation(
            JFrame.EXIT_ON_CLOSE);
        ventana.setVisible(true);
    }
}
```

BUENAS PRÁCTICAS

Usar lambdas más limpio que clases anónimas	Validar entradas parseInt puede fallar → try/catch
Separar lógica de GUI patrón MVC en proyectos grandes	Usar JPanel nunca añadir todo directo al JFrame
setVisible(true) al final siempre la última instrucción	No saturar el main extraer métodos para cada parte

TABLA RESUMEN DEL BLOQUE

Operación	Clase / Método
Crear ventana	new JFrame("titulo")
Mostrar ventana	setVisible(true)
Cerrar al salir	setDefaultCloseOperation(EXIT_ON_CLOSE)
Crear botón	new JButton("texto")
Asignar evento	boton.addActionListener(e → {...})
Campo de texto	new JTextField(cols)
Leer campo	campo.getText()
Escribir campo	campo.setText("val")
Agrupar componentes	new JPanel() + panel.add(comp)
Mensaje emergente	JOptionPane.showMessageDialog(null, "msg")
Pedir dato	JOptionPane.showInputDialog("msg")
Confirmar	JOptionPane.showConfirmDialog(null, "msg")

Patrón mínimo: `JFrame` → `JPanel` → componentes → `addActionListener(lambda)` → `setVisible(true)`. Valida siempre el texto antes de parsear.